

Problem Statement #49

Webhook Ingestion & Retry Mechanism Hub

A resilient webhook gateway receiving third-party events, logging request payloads, delivering to internal services, and managing dead-letter queues with exponential backoff retries.

Target Stakeholders / Actors: Integration Developer, System Operator, Third-Party Application

Contents

1. Requirements Table (FR-001–FR-005, NFR-001–NFR-002)
2. UML Use-Case Diagram
3. Use-Case Flow Specification

1. Requirements Table

Functional Requirements

ID	Priority	Description	Acceptance Criteria	Rationale
FR-001	High	The system shall receive inbound webhooks, forward payloads to the configured target endpoint, and automatically retry failed deliveries using exponential backoff (1s, 2s, 4s, 8s).	Pass: Failed delivery successfully retried and logged. Fail: Dropped webhook without dead-letter persistence.	Third-party senders cannot be relied upon to resend failed events, so the gateway must guarantee delivery attempts on their behalf.
FR-002	High	The system shall log every received webhook, persisting payload, headers, source, and timestamp immediately after receipt, before any processing.	Pass: A log record exists immediately after receipt, even if forwarding later fails. Fail: A payload is lost or unrecoverable after a crash.	Raw payloads must be recoverable for debugging, audit, and replay regardless of downstream failures.
FR-003	High	The system shall forward each logged webhook to the correct internal target endpoint based on the source/event-type configuration.	Pass: Payload is delivered to the endpoint matching its configured route. Fail: Payload is sent to the wrong endpoint or dropped due to missing route.	Multiple integrations may share one gateway, so correct routing is essential to avoid cross-delivery of events.
FR-004	High	If forwarding a webhook fails, the system shall retry the failed delivery using exponential backoff, and shall manage the dead-letter queue by moving the webhook there once the maximum retries are exhausted.	Pass: Webhook appears in the Dead-Letter Queue only after the final retry fails, with full failure history recorded. Fail: Webhook is dropped silently or retried indefinitely.	Permanently failed deliveries must not be lost; operators need visibility and a recovery path once the downstream issue is fixed.
FR-005	Medium	The system shall allow the Integration Developer to configure target endpoints and shall allow monitoring of webhook delivery status by both the Integration Developer and System Operator.	Pass: A newly configured endpoint begins receiving forwarded payloads within one polling cycle, and current delivery status is visible on demand. Fail: Configuration changes are not reflected in routing, or status is unavailable.	Self-service configuration and visibility reduce operational overhead and let integrations be onboarded without code changes.

Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Security	The ingestion endpoint shall buffer inbound webhooks with 99.999% reliability and throughput up to 2,500 requests/second.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load. Fail: Requests are dropped or latency exceeds threshold.	High-volume senders can burst traffic; the gateway must not become a bottleneck or single point of failure.
NFR-002	Reliability & Availability	The system shall guarantee 99.9% monthly uptime for the ingestion endpoint and shall not lose any logged webhook during a service restart or crash.	High	Pass: Chaos/restart testing shows zero data loss and uptime monitoring confirms the 99.9% target over a rolling 30-day window. Fail: Any logged webhook is missing after a restart.	Webhook senders typically do not retry indefinitely themselves, so the gateway's own availability directly determines whether events are captured.

2. UML Use-Case Diagram

Actors: Third-Party Application, Integration Developer, System Operator. The diagram includes one <<include>> relationship (Receive Webhook includes Log Webhook; Log Webhook includes Forward Webhook) and one <<extend>> relationship (Retry Failed Delivery extends Forward Webhook, firing only when a delivery attempt fails).



3. Use-Case Flow Specification

Use Case: Retry Failed Delivery

Actors: System Operator (secondary), Integration Developer (secondary)

System: Webhook Ingestion & Retry Mechanism Hub

Brief Description

This use case extends Forward Webhook: when an attempt to forward a logged webhook to its target endpoint fails, the system automatically retries delivery using an exponential backoff schedule until it either succeeds or exhausts the configured maximum number of attempts, at which point it includes Manage Dead-Letter Queue to record and escalate the failure.

Preconditions

1. A webhook has already been received and logged by the system (Receive Webhook, Log Webhook).
2. The webhook has been routed to a configured internal target endpoint (Forward Webhook).
3. The initial delivery attempt to that endpoint has failed (timeout, non-2xx response, or connection error), triggering this extension.

Postconditions

Success path: The webhook is marked DELIVERED, the delivery timestamp and attempt count are logged, and no further retries occur.

Failure path: The webhook is marked DEAD_LETTERED via Manage Dead-Letter Queue, and the System Operator is notified.

Main Success Scenario

1. The system detects that the delivery attempt to the target endpoint has failed and records the failure reason.
2. The system checks the current retry count against the maximum allowed retries (default: 4).
3. The system calculates the next backoff interval using the exponential schedule (1s, 2s, 4s, 8s) based on the current attempt number.
4. The system waits for the calculated interval, then re-attempts delivery to the same target endpoint.
5. The target endpoint returns a success response (HTTP 2xx).
6. The system marks the webhook as DELIVERED, logs the successful attempt number and timestamp, and stops the retry cycle.

Alternate Flow — Maximum Retries Exhausted

Branches from step 4 when the retry attempt also fails.

- 4a. The re-attempted delivery fails again (timeout, non-2xx response, or connection error).
- 4b. The system increments the retry counter and checks it against the maximum retry limit.
- 4c. If the counter has not reached the limit, the flow returns to step 3 and schedules the next backoff interval.
- 4d. If the counter has reached the limit (4 failed attempts), the system includes Manage Dead-Letter Queue, which persists the full failure history and marks the webhook DEAD_LETTERED.
- 4e. The System Operator is notified and may later trigger a manual replay of the dead-lettered webhook, which resets the retry counter and returns the flow to step 1.

Business Rules

Backoff intervals are fixed at 1s, 2s, 4s, 8s. A webhook is never silently dropped: it always ends in either DELIVERED or DEAD_LETTERED state.

Special Requirements

Retry timing must not block ingestion of new incoming webhooks (retries run asynchronously). All state transitions must be logged for audit purposes (supports NFR-002).